# GameStop Collectibles Dynamic Pricing Engine

Technical Solution Architecture & Implementation Plan

**Prepared by:** Ish Prasad | Staff AI/ML Engineer **Date:** April 2026 **For:** Ravi & Team — GameStop AI/ML Engineering

## Executive Summary

GameStop's collectibles business (graded trading cards — Pokemon, sports, TCG) currently prices cards using daily batch scrapes from eBay, with no price history storage, no sentiment analysis, and no ML-based pricing. This document presents a complete technical architecture for a **dynamic pricing engine** that combines multi-source market data, social sentiment, ML price prediction with uncertainty quantification, and anomaly detection — all designed to be built in-house with open-source tools.

**The core insight:** Collectibles pricing is a *sparse, irregular, fat-tailed* prediction problem — not a standard time series. Most cards sell infrequently (1-10x per month), prices can 10x overnight on social virality, and the same card at different grades can differ 50x in value. The system must handle this heterogeneity with volume-aware routing, not a one-size-fits-all model.

**GameStop's unique advantage:** Physical store foot traffic + trade-in pricing data that no online competitor (PriceCharting, Alt, Card Ladder) has access to. This is the moat.

# 1. Data Architecture

## 1.1 Data Sources — Priority Ranked

| PRIORITY | SOURCE | DATA TYPE | ACCESS METHOD | COST | UPDATE FREQUENCY |
|---|---|---|---|---|---|
| **P0** | eBay Marketplace Insights API | Sold listings (90 days) | Limited Release API — negotiate access | Negotiate | Event-driven (every 15 min) |
| **P0** | SoldComps API | eBay sold listings (365 days) | REST API | $59/mo (5,000 req/mo) | Every 15 min (cron) |
| **P0** | PriceCharting API | Aggregated prices (all time) | REST API (40-char token) | ~$15-30/mo (Legendary tier) | Daily (2 AM cron) |
| **P1** | PSA Population Reports | Grade population counts | Web scrape (psacard.com/pop) or GemRate | Free (GemRate) | Daily (3 AM cron) |
| **P1** | CardHedger API | 40M+ transactions, multi-platform | REST API | $49+/mo | Near real-time |
| **P1** | Reddit (old.reddit.com) | Social sentiment | Browser Rendering scrape (old.reddit.com) | Free (Cloudflare Workers) | Every 5 min (cron) |
| **P2** | Twitter/X API | Event detection, viral moments | Pay-per-use | ~$500-1K/mo | **DEFERRED** |
| **P2** | TCGPlayer API | Pokemon/MTG market prices | Apply for access | Free (with approval) | Daily |
| **P2** | PokemonPriceTracker API | Graded Pokemon prices + pop | REST API | $99/mo (Business) | Daily |
| **P3** | GameStop internal | Trade-in data, foot traffic, inventory | Internal DB | Free | **DEFERRED** — requires integration |

## 1.2 eBay Strategy — Legal & Practical

**Do NOT scrape eBay directly.** eBay explicitly bans AI agents and scraping in their Feb 2026 User Agreement. As a publicly traded company with an eBay business relationship, GameStop cannot afford the legal/business risk.

**Instead:** 1. **Negotiate Marketplace Insights API access** — GameStop's retail footprint gives leverage. This API returns sold items for last 90 days and is the gold standard. 2. **Use SoldComps as fallback** ($59/mo) — licensed eBay sold data via REST API, up to 240 results per request, 365 days history. 3. **Use CardHedger for multi-platform** ($49/mo) — aggregates eBay, Fanatics, Heritage Auctions, and more.

## 1.3 Price History Storage

GameStop doesn't store price history. This is the foundation everything else depends on.

**Built on Cloudflare D1** (SQLite at the edge, zero cold starts, global replication, ~$5-20/mo at moderate scale):

```
Tables (all implemented):
├── card_catalog        — master card registry (id, name, set_name, set_year, card_number,
│                         category, player_character, team, rarity, image_url, pricecharting_id)
│                         Categories: pokemon, sports_baseball, sports_basketball,
│                         sports_football, sports_hockey, tcg_mtg, tcg_yugioh, other
├── price_observations  — every API price point
│   (card_id, source, price_usd, sale_date, grade, grading_company, grade_numeric,
│    sale_type, seller_id, bid_count, listing_url, is_anomaly, anomaly_reason)
│   Sources: ebay, soldcomps, pricecharting, cardhedger, tcgplayer, gamestop_internal
│   Dedup: unique index on (card_id, source, listing_url)
├── price_aggregates    — daily/weekly/monthly rollups per card+grade
│   (avg_price, median_price, min_price, max_price, sale_count, volume_bucket)
├── population_reports  — PSA/CGC/BGS/SGC pop snapshots
├── sentiment_raw       — individual Reddit/Twitter observations
├── sentiment_scores    — rolled-up 24h/7d/30d sentiment scores
├── feature_store       — pre-computed ML features per card
├── model_predictions   — model outputs with confidence bands
│   (fair_value, p10, p25, p50, p75, p90, buy_threshold, sell_threshold,
│    confidence [HIGH/MEDIUM/LOW], volume_bucket)
├── price_alerts        — triggered alerts (price_spike, price_crash, viral_social,
│                         anomaly_detected, new_high, new_low)
└── ingestion_log       — pipeline run tracking (source, status, records_processed, error_message)

Archival: old observations moved to R2 (Parquet/CSV)
D1 limit: 10GB per database — plan for archival at ~6 months
```

### 1.4 Data Pipeline Architecture

The entire pipeline runs on **Cloudflare Cron Triggers**, **Cloudflare Queues**, and **Durable Object agents**. There is no Airflow, no separate orchestrator.

```
                    ┌──────────┐
                    │  Sources │
                    └──────────┘
                         │
       ┌─────────────────┼─────────────────┐
       │                 │                 │
  eBay/SoldComps    Reddit scrape      PriceCharting
  (every 15 min)    (every 5 min,     (daily 2am)
                     rotates subreddits)
       │                 │                 │
       ▼                 ▼                 ▼
  ┌──────────────────────────────────────────┐
  │     Cloudflare Cron Triggers + Queues    │
  │                                          │
  │  */15 * * * *  → SoldComps ingestion     │
  │  */5  * * * *  → Reddit sentiment        │
  │  0   * * * *   → Sentiment rollup (hourly)│
  │  0 2 * * *     → PriceCharting           │
  │  0 3 * * *     → PSA population reports  │
  │  0 4 * * *     → Anomaly detection       │
  │  0 5 * * *     → Aggregates + features   │
  │  0 6 * * *     → Batch predictions       │
  │                                          │
  │  Queue: gamecards-ingestion (async batch)│
  │  Queue: gamecards-sentiment (async AI)   │
  └──────────────────────────────────────────┘
                    │
                    ▼
  ┌──────────────────────────────────────────┐
  │       Cloudflare D1 (Price History)      │
  │  + KV (cache hot prices, rate limiting)  │
  │  + R2 (raw data archive, model artifacts)│
  │  + Workers AI (sentiment NLP — Gemma 4 26B)│
  └──────────────────────────────────────────┘
                    │
          ┌─────────┼─────────┐
          │         │         │
          ▼         ▼         ▼
     Pricing API  Dashboard  Agents (DOs)
     (Hono/Workers) (Vite/React) (4 agents)
```

**Pipeline ordering is critical** — anomaly detection (4 AM) must run before features/predictions (5-6 AM) so flagged outliers are excluded from downstream computation.

# 2. Feature Engineering

## 2.1 Feature Categories & Importance

Based on research across collectibles pricing systems, academic papers, and production implementations (PriceCharting, Alt, Card Ladder, StockX):

| RANK | FEATURE GROUP | PREDICTIVE POWER | IMPLEMENTATION COMPLEXITY |
|---|---|---|---|
| 1 | **Grade + Grading Company** | ~25-30% of variance | Low — structured data from PSA/CGC |
| 2 | **Card Identity** (set, number, player/character) | ~20-25% | Medium — learned embeddings |
| 3 | **Recent Sold Prices** (lagged features) | ~15-20% | Low — rolling stats from price history |
| 4 | **Population Report** (supply) | ~8-12% | Medium — daily pop scraping |
| 5 | **Sold Velocity** (demand signal) | ~5-8% | Low — count sales per window |
| 6 | **Macro Market Index** | ~3-5% | Medium — aggregate market health |
| 7 | **Player/Meta Performance** | ~3-5% | High — domain-specific APIs |
| 8 | **Seasonality** | ~2-3% | Low — calendar features |
| 9 | **Social Sentiment** | ~1-3% | High — NLP pipeline required |
| 10 | **Rarity Metrics** | ~2-4% | Medium — print run data |

## 2.2 Critical Feature Details

**Grade & Population Features:**

```
{
    'grade_numeric': 9.5,                   # 1-10 scale
    'grading_company': 'PSA',               # PSA, BGS, CGC, SGC
    'is_gem_mint': True,                    # grade ≥ 9.5
    'is_perfect_10': False,
    'pop_at_grade': 47,                     # how many exist at this grade
    'pop_higher': 3,                        # how many graded higher
    'pop_ratio': 0.047,                     # % of total pop at this grade
    'is_pop_1': False,                      # unique at this grade
    'pop_growth_rate_90d': 0.12,            # 12% more graded in last 90 days
    # KEY INSIGHT: PSA 10 with pop 5 is worth dramatically more than pop 500
    # The relationship is highly nonlinear — tree models capture this well
}
```

**Demand Signals:** `sales_count_7d/30d/90d`, `velocity_trend` (7d/30d ratio), `price_momentum` (7d MA / 30d MA).

> **DEFERRED features** (require live listing snapshots): `active_listings_count`, `days_of_inventory`, `sell_through_rate`, `avg_bidders_last_5_auctions`. Need eBay Browse API + listing-snapshot pipeline.

**Sentiment Features (from Reddit pipeline):** `social_mention_count_7d`, `social_mention_trend` (7d/30d ratio), `social_sentiment_score` (-1 to 1), `reddit_post_count_7d`, `influencer_mention_7d`, `search_interest_trend`.

**Seasonality Features:** `is_holiday_season`, `is_tax_refund_season`, `is_summer_lull`, `is_nfl_season`, `days_since_set_release`, cyclical month encoding (`month_sin`, `month_cos`).

**GameStop-Exclusive Features (the moat) — DEFERRED:** `gamestop_trade_in_volume_7d`, `gamestop_store_price`, `gamestop_inventory_count`, `gamestop_days_in_inventory`, `gamestop_regional_demand`. No competitor has this data — requires internal DB integration.

---

# 3. ML Model Architecture

## 3.1 The Core Problem: Heterogeneous Liquidity

Not all cards are created equal. A base set Charizard PSA 10 sells multiple times per week. A 1987 Topps Barry Bonds PSA 8 sells once every 2 months. The model architecture must adapt to volume.

## 3.2 Volume-Aware Ensemble

```
┌──────────────────────────────────────────────────────────┐
│                    ENSEMBLE PRICING ENGINE                  │
├──────────────────────────────────────────────────────────┤
│                                                            │
│  HIGH VOLUME (50+ sales/quarter)           ~15% of cards    │
│  ├── LightGBM quantile regression (primary)                 │
│  ├── Temporal Fusion Transformer (temporal dynamics)         │
│  └── LSTM for momentum capture                              │
│  → Meta-learner weighted average                            │
│  → Tight confidence intervals (±10-15%)                     │
│                                                             │
│  MEDIUM VOLUME (10-50 sales/quarter)       ~25% of cards    │
│  ├── LightGBM with stronger regularization                  │
│  ├── Gaussian Process regression (natural uncertainty)       │
│  └── KNN comparable sales (cold-start backup)               │
│  → GP uncertainty widens intervals (±15-25%)                │
│                                                             │
│  LOW VOLUME (< 10 sales/quarter)           ~60% of cards    │
│  ├── Hierarchical Bayesian model (borrows from similar)     │
│  ├── GP with informative priors                             │
│  └── Comparable sales (KNN in feature space)                │
│  → Wide uncertainty bands (±25-40%), flag for human review  │
│                                                             │
│  MARKET INDEX (feeds into all above as a feature)           │
│  └── Prophet decomposition on aggregate market data         │
│                                                             │
│  META-LEARNER                                               │
│  └── Stacking regressor: learns which base model to trust   │
│      based on volume, price range, volatility               │
└──────────────────────────────────────────────────────────┘
```

## 3.3 Primary Model: LightGBM Quantile Regression

LightGBM is the right primary model because: - Handles mixed feature types natively (categorical grades, continuous prices) - Captures nonlinear grade-price relationships (PSA 9 vs 10 can be 5-10x) - Fast inference (<1ms per card) - Handles missing features gracefully (many features will be NaN for low-volume cards) - Interpretable via SHAP values

```python
import lightgbm as lgb

# Train on log(price) — normalizes fat-tailed distribution
params = {
    'objective': 'huber',           # Overridden to 'quantile' in the loop below
    'metric': 'mae',
    'learning_rate': 0.03,
    'num_leaves': 63,
    'min_data_in_leaf': 20,
    'feature_fraction': 0.8,
    'bagging_fraction': 0.8,
    'bagging_freq': 5,
    'lambda_l1': 0.1,
    'lambda_l2': 1.0,
    'verbose': -1,
}

# Quantile models for uncertainty — includes p20/p80 for buy/sell thresholds
quantiles = [0.10, 0.20, 0.25, 0.50, 0.75, 0.80, 0.90]
models = {}
for q in quantiles:
    params_q = {**params, 'objective': 'quantile', 'alpha': q}
    models[q] = lgb.train(params_q, train_set, num_boost_round=2000,
                          valid_sets=[val_set],
                          callbacks=[lgb.early_stopping(50)])

# Inference: full price distribution
fair_value = np.exp(models[0.50].predict(X))      # Median = fair market value
buy_threshold = np.exp(models[0.20].predict(X))    # Buy if below this
sell_threshold = np.exp(models[0.80].predict(X))   # Sell if above this
confidence_width = (np.exp(models[0.90].predict(X)) - np.exp(models[0.10].predict(X))) / fair_value
```

**Why Huber loss, not MSE:** A single anomalous $3,000 sale of a card that normally trades at $50 will destroy a model trained with MSE. Huber loss is robust to these fat-tailed outliers.

**Why log(price):** Collectibles prices are log-normally distributed. A $50 card varying +/-$10 is the same relative uncertainty as a $5,000 card varying +/-$1,000.

### 3.4 Conformal Prediction for Calibrated Uncertainty

Quantile regression gives intervals, but they may not be calibrated (the 90% interval might only cover 78% of actuals). Conformal prediction wraps any model to give **distribution-free coverage guarantees** by computing nonconformity scores on a calibration set and using the empirical quantile as the interval width.

### 3.5 Buy/Sell Decision Framework

> **Critical:** Thresholds are based on **net realizable value (NRV)** — what GameStop actually nets after selling — not raw market quantiles.

Economics constants (implemented in `PricingRecommendationAgent`): marketplace fee 13%, shipping $5, return rate 3%, required margin 20%. NRV = `fair_value * (1 - fee) * (1 - return_rate) - shipping`. Max buy price = `NRV * (1 - required_margin)`.

# 4. Sentiment Analysis Pipeline

## 4.1 Architecture

```
Reddit via Browser Rendering (old.reddit.com/.json)
        |
        ▼
┌──────────────────────────────────────┐
|  NLP Pipeline (Workers AI)            |
|                                      |
|  1. Card Mention Extraction (NER)     |
|     "That Charizard PSA 10 is fire"   |
|     → card: Charizard, grade: PSA 10  |
|                                      |
|  2. Sentiment Classification          |
|     Workers AI (Gemma 4 26B +         |
|     DistilBERT) at the edge           |
|     → bullish (0.85)                  |
|                                      |
|  3. Hype Detection                    |
|     Volume spike + positive sentiment |
|     → HYPE_ALERT: Charizard PSA 10    |
|                                      |
|  4. Aggregation (hourly cron)         |
|     Rolling 24h / 7d / 30d scores     |
|     Weighted by engagement (upvotes)  |
└──────────────────────────────────────┘
                  |
                  |
                  ▼
         D1 sentiment_scores table
         → Feeds into ML feature pipeline
```

**Note:** Reddit deprecated its Data API in favor of Devvit, so we scrape via Cloudflare's headless Chrome (Browser Rendering) instead of using the API directly. We fetch `old.reddit.com/<subreddit>.json` through Browser Rendering, which returns structured JSON without requiring API credentials.

**Note:** Twitter/X integration is **DEFERRED**. v1 uses Reddit only.

## 4.2 Target Subreddits

| SUBREDDIT | MEMBERS | FOCUS | EST. DAILY VOLUME |
|---|---|---|---|
| r/PokemonTCG | ~280K | Pokemon cards | ~500 posts/day |
| r/baseballcards | ~150K | Baseball | ~300 posts/day |
| r/basketballcards | ~60K | Basketball | ~150 posts/day |
| r/footballcards | ~50K | Football | ~120 posts/day |
| r/PKMNTCGDeals | ~100K | Pokemon deals/pricing | ~200 posts/day |
| r/sportscards | ~30K | General sports | ~80 posts/day |
| r/TradingCardCommunity | ~15K | General TCG | ~50 posts/day |

## 4.3 Viral Event Detection

Social media sentiment's biggest value is **catching viral moments** before they hit prices. The **PriceMonitorAgent** implements this: if social mentions in a 6-hour window exceed 3x the 7-day rolling average, it fires a `viral_social` alert and invalidates the KV price

cache for affected cards.

---

# 5. Anomaly Detection

## 5.1 Seller-Side Anomaly Detection

**Data limitation:** SoldComps, CardHedger, and PriceCharting return sold-listing summaries, not bid-level event data. True shill-bidding detection is not feasible without a bid-history feed.

```
What we CAN detect from sold listings:
├── Seller price inflation — sellers consistently above market (via seller_id if available)
├── Statistical price outliers — IQR-based detection on price vs. historical average
├── Suspicious pricing patterns — high-value graded cards at <$1 (data quality)
└── Price spike/crash detection — 7d vs 30d moving average divergence > 30%

What we CANNOT detect without bid-event data:
├── Same bidder repeatedly losing to same seller
├── Bid increment manipulation
├── Concentrated bidding in final seconds
└── Buyer-seller relationship graphs

Model: Statistical outlier detection + seller concentration (if seller_id available)
Action: Flag price_observations.is_anomaly = true, exclude from training/features
```

Anomaly detection runs daily at **4 AM UTC** (before features at 5 AM, predictions at 6 AM) so flagged outliers are excluded from downstream computation.

## 5.2 Data Quality Issues

- **"Best Offer Accepted" bias:** eBay shows listing price, not accepted price. Biases upward 15-25%. Fix: flag and discount by 20%, or exclude.
- **Lot sales:** Filter via keyword detection (lot, bundle, collection, set of, x2, x3).
- **Currency normalization:** International sales in GBP/EUR/AUD must be converted to USD at the sale date exchange rate.

# 6. Backtesting & Evaluation

## 6.1 Walk-Forward Validation

**Never use random train/test splits for time series data.** Always train on past, test on future. Full walk-forward requires 6+ months of raw transaction data. At launch, use PriceCharting backfill with reduced feature set.

## 6.2 Metrics

| METRIC | TARGET |
|---|---|
| **MdAPE** (Median Absolute % Error) | <15% high-vol, <25% mid-vol, <40% low-vol |
| **Directional Accuracy** | >60% |
| **Coverage** (90% CI) | 87-93% |
| **Interval Width %** | <30% for high-vol |
| **Simulated Trading P&L** | Positive over backtest |

**Critical:** Report metrics **stratified by volume bucket.**

## 6.3 Monitoring Price Floor

Sub-$10 sales (accessories, commons, mismatches) create astronomical percentage errors that don't reflect model quality. A $2 card predicted at $4 is a 100% error but only $2 off — not a real pricing failure. MAPE (trimmed) computed on $10+ sales is **42% with p10-p90 coverage of 91%**. All production monitoring dashboards apply a $10 price floor filter.

## 6.4 Known ML Evaluation Limitations

Two structural limitations affect offline evaluation quality:

**1. Point-in-time feature leakage.** `feature_store` contains only the latest snapshot per card (one row, updated in place). When exporting training data, all historical sales are joined to today's features — meaning a 2023 sale gets 2026 velocity, momentum, and seasonality values. This overstates model accuracy for temporal features (~30% of signal weight). Static features (grade, population, card identity — ~70% of signal) are unaffected. **Fix:** Store daily feature snapshots and join each sale to the features as of its sale date. Deferred until data volume justifies storage cost (~250K rows/year).

**2. Backtest P&L uses the target as the decision input.** The simulated trading function compares each sale's actual price against the model's buy/sell thresholds. In production, the "offered price" comes from a customer or listing, not from the test set target. This means the backtest P&L metric is directionally indicative but not a clean simulation of the production buy/sell policy. **Fix:** Generate a separate synthetic "offer stream" for backtesting, or use a held-out set of recent trade-in transactions as the offer source.

**Impact:** The reported 42% MAPE should be treated as an approximate lower bound. True out-of-sample accuracy with proper point-in-time features may be higher (worse). The model is still production-useful for directional signals and relative ranking, but the exact numeric accuracy claim requires the fixes above.

# 7. System Architecture & Infrastructure

## 7.1 Technology Stack

| LAYER | TECHNOLOGY | STATUS |
|---|---|---|
| **Runtime** | Cloudflare Workers (Hono framework) | **Shipped** |
| **Data Storage** | Cloudflare D1 (SQLite at edge) | **Shipped** |
| **Cache** | Cloudflare KV (hot price lookups, rate limiting) | **Shipped** |
| **Object Storage** | Cloudflare R2 (data archive, model artifacts) | **Shipped** |
| **Queues** | Cloudflare Queues (gamecards-ingestion, gamecards-sentiment) | **Shipped** |
| **Orchestration** | Cloudflare Cron Triggers (8 schedules) | **Shipped** |
| **Agents** | Cloudflare Durable Objects (Agents SDK) | **Shipped** — 4 agents |
| **NLP** | Workers AI (Gemma 4 26B + DistilBERT) | **Shipped** |
| **Browser Rendering** | Cloudflare headless Chrome | **Shipped** — Reddit scraping without API |
| **ML Training** | LightGBM + scikit-learn via Cloudflare Containers | **Shipped** |
| **Containers** | Cloudflare Containers — Python ML training on-demand | **Shipped** — Train → score → upload to R2, no external compute |
| **ML Export** | ONNX via onnxmltools/skl2onnx → R2 | **Shipped** |
| **ML Serving** | LightGBM quantile regression (707 cards) + statistical fallback for new cards | **Shipped** |
| **Experiment Tracking** | MLflow (local/self-hosted) | **Shipped** |
| **Frontend** | Vite + React + Tailwind v4 + React Router | **Shipped** |
| **Auth** | API key middleware on `/v1/*` + agent routes | **Shipped** |
| **CI** | GitHub Actions (typecheck, lint, Python syntax) | **Shipped** |
| **Retraining** | GitHub Actions weekly cron (Sunday 2 AM UTC) via Cloudflare Containers | **Shipped** |
| **Monitoring** | Cloudflare Analytics + `ingestion_log` table | **Shipped** |
| **A/B Testing** | Model comparison framework | **DEFERRED** |
| **GameStop Integration** | Internal DB for trade-in/inventory data | **DEFERRED** |

### 7.1.1 Security & Authentication

Implemented: - **SignIn gate:** Dashboard requires an access code entry on a SignIn page. The code is server-validated against the `DEMO_ACCESS_CODE` secret. - **Session tokens:** `POST /v1/auth/login` accepts `{ code }` and returns an HMAC-signed session token with 24-hour expiry, stored in KV. - **Auth middleware:** All `/v1/*` routes accept either a static API key ( `X-API-Key` header) OR a session token ( `Authorization: Bearer <token>` ). Bypassed in development environment. - **Session management:** `POST /v1/auth/verify` checks token validity and returns `{ valid, expiresAt }` . `POST /v1/auth/logout` invalidates the session token in KV. - **Agent auth:** All `/agents/*` routes also gated by the same auth middleware in non-development environments. - **CORS:** Locked to allowed origins, configurable via `ALLOWED_ORIGINS` environment variable. - **Rate limiting:** KV-based throttling at 120 requests/min per token on `/v1/*` . - **Secrets management:** All API keys stored via `wrangler secret put` — `SOLDCOMPS_API_KEY` , `PRICECHARTING_API_KEY` ,

`CARDHEDGER_API_KEY`, `POKEMON_PRICE_TRACKER_KEY`, `API_KEY`, `DEMO_ACCESS_CODE`, `SESSION_SECRET`. - **D1 access:** Restricted to Workers bindings only — no public database endpoint.

Required before production: - **GameStop SSO:** Dashboard should require GameStop SSO for production. - **Data privacy:** `seller_id` should be hashed before storage.

### 7.1.2 Custom Domains & Deployment

- **API:** `api.gmestart.com` — Cloudflare Worker (Hono)
- **Dashboard:** `app.gmestart.com` — Cloudflare Pages (Vite + React)
- **SignIn gate:** Dashboard requires access code `GMESTART2026` before granting a session token. The code is validated server-side via `POST /v1/auth/login`.

## 7.2 API Endpoints

All endpoints are under `/v1/` and require API key or session token authentication (except the root health check and auth endpoints).

**Auth:**

```
POST /v1/auth/login               — { code } → { token, expiresAt }
POST /v1/auth/verify              — → { valid, expiresAt }
POST /v1/auth/logout              — → { status }
```

**Root Health Check:**

```
GET  /
     → { service, version, status: "healthy"|"degraded", environment,
         auth, predictions: "fresh"|"stale" }
     Checks model_predictions freshness (stale if >36 hours old).
```

**Cards:**

```
GET  /v1/cards                    — List/search card catalog
GET  /v1/cards/:id                — Get single card details
```

**Prices:**

```
GET  /v1/price/:card_id?grade=&grading_company=
     → { card_id, card_name, grade, grading_company, price, lower, upper,
         buy_threshold, sell_threshold, confidence, last_sale, sales_30d,
         trend, updated_at, has_prediction }
     Returns model prediction if available, statistical fallback otherwise.
```

**History:**

```
GET  /v1/history/:card_id?grade=&days=
     → [{ date, price, source, sale_type }, ...]
```

**Sentiment:**

```
GET  /v1/sentiment/:card_id
     → { score, mention_count, period, top_posts, computed_at }
```

**Evaluate (Buy/Sell Decision):**

```
POST /v1/evaluate
     Body: { card_id, offered_price, grade?, grading_company? }
     → { decision: "STRONG_BUY"|"REVIEW_BUY"|"FAIR_VALUE"|"SELL_SIGNAL",
         fair_value, margin, confidence, reasoning }
```

**Alerts:**

```
GET  /v1/alerts                  — Active alerts
POST /v1/alerts/:id/resolve      — Resolve an alert
```

**Market:**

```
GET  /v1/market/index
     → { pokemon_index, sports_index, trend_30d, volatility, updated_at }
```

**System (operational):**

```
GET  /v1/system/health           — Pipeline health (prediction freshness, model version,
                                   ingestion status, catalog count)
GET  /v1/system/model            — Current model metadata from R2
POST /v1/system/rollback         — Full rollback: copies versioned predictions from R2,
                                   re-imports into D1, invalidates KV cache, updates meta
     Body: { version_key: "models/versions/..." }
POST /v1/system/bootstrap        — Bootstrap card catalog from PriceCharting
POST /v1/system/seed             — Seed database with sample data for development/demo
POST /v1/system/mock-internal    — Generate mock internal GameStop data for testing
```

**Agents (REST proxy for Durable Object @callable methods):**

```
GET  /v1/agents/monitor/status
POST /v1/agents/monitor/check

GET  /v1/agents/intelligence/latest
POST /v1/agents/intelligence/generate

GET  /v1/agents/competitors/status
GET  /v1/agents/competitors/gaps
POST /v1/agents/competitors/scan

GET  /v1/agents/recommendations/pending?action=BUY|SELL|REPRICE
GET  /v1/agents/recommendations/status
POST /v1/agents/recommendations/generate
POST /v1/agents/recommendations/:id/approve   Body: { approvedBy? }
POST /v1/agents/recommendations/:id/reject    Body: { rejectedBy? }
```

Agents are also accessible via WebSocket using the Agents SDK `useAgent()` hook from the dashboard.

## 7.3 Autonomous Agents (Durable Objects)

Four Durable Object agents run autonomously alongside the cron pipeline. Each uses the Cloudflare Agents SDK with hibernation, automatic retry, and real-time state sync via WebSocket.

### 7.3.1 PriceMonitorAgent

**Purpose:** Detects price anomalies and viral social events in real time, triggers cache invalidation for affected cards.

| PROPERTY | VALUE |
|---|---|
| Schedule | `scheduleEvery(900)` — every 15 minutes |
| Hibernate | Yes |
| Retry | 3 attempts, 1-30s backoff |
| Max alerts | 100 (validated on state change) |

**State:** `lastCheckAt`, `activeAlerts[]`, `checksRun`, `anomaliesDetected`

**Callable methods:** - `runMonitoringCheck()` — Queries D1 for 30% 1d-vs-30d price divergence + viral social mentions (>3× 7d average in 6h). Writes `price_alerts` rows. Invalidates KV cache for affected cards. - `getStatus()` — Returns last check time, alert counts, recent alerts. - `clearAlerts()` — Clears active alerts. - `listSchedules()` / `cancelTask(id)` — Schedule management.

### 7.3.2 MarketIntelligenceAgent

**Purpose:** Generates daily AI-powered market briefings using Workers AI (Gemma 4 26B).

| PROPERTY | VALUE |
|---|---|
| Schedule | `schedule("0 7 * * *")` — daily at 7 AM |
| Hibernate | Yes |
| Retry | 2 attempts, 5-60s backoff |
| Reports stored | Last 30 |

**State:** `reports[]`, `lastGeneratedAt`, `totalReports`

**Callable methods:** - `generateDailyReport()` — Queries D1 for top gainers/decliners (7d), active alerts, sentiment summary, volume stats. Sends to Gemma 4 26B for prose summary. Returns `MarketReport` with highlights, top movers, and market sentiment (bullish/bearish/neutral). - `getLatestReport()` — Most recent report. - `getReportHistory(count)` — Last N reports (default 7). - `getStatus()` — Generation stats.

### 7.3.3 CompetitorTrackerAgent

**Purpose:** Scans for price gaps between GameStop's fair value and competitor prices (PriceCharting, CardHedger, SoldComps).

| PROPERTY | VALUE |
|---|---|
| Schedule | `scheduleEvery(21600)` — every 6 hours |
| Hibernate | Yes |
| Retry | 3 attempts, 2-30s backoff |
| Max gaps tracked | 50 |

**State:** `lastScanAt`, `priceGaps[]`, `scansCompleted`, `opportunitiesFound`

**Callable methods:** - `scanCompetitorPrices()` — Compares top 100 cards' `model_predictions.fair_value` against recent `price_observations` from external sources. Flags gaps >15%. Matches on grade+grading_company to avoid false gaps. - `getOverpriced(limit)` / `getUnderpriced(limit)` — Filtered gap lists. - `getAllGaps()` — All gaps sorted by magnitude. - `getStatus()` — Scan stats with overpriced/underpriced counts.

### 7.3.4 PricingRecommendationAgent

**Purpose:** Generates actionable BUY/SELL/REPRICE recommendations with human-in-the-loop approval workflow.

| PROPERTY | VALUE |
|---|---|
| Schedule | `schedule("0 8 * * *")` — daily at 8 AM (recommendations) |
| | `scheduleEvery(21600)` — every 6h (expire stale) |
| Hibernate | Yes |
| Retry | 2 attempts, 5-60s backoff |
| Max pending | 200 (validated on state change) |

**State:** `pending[]`, `history[]`, `lastGeneratedAt`, `stats` (totalGenerated/Approved/Rejected/Expired)

**Economics constants:** `MARKETPLACE_FEE = 13%`, `SHIPPING_COST = $5`, `RETURN_RATE = 3%`, `REQUIRED_MARGIN = 20%`

**Callable methods:** - `generateRecommendations()` — For all HIGH/MEDIUM confidence predictions with fair_value > $10: computes NRV, generates BUY (market < max buy price), SELL (market > sell threshold), or REPRICE (near breakeven). Top 50 by margin, sorted. - `approveRecommendation(id, approvedBy)` — Moves to history as approved. - `rejectRecommendation(id, rejectedBy)` — Moves to history as rejected. - `getPending(action?)` — Filter pending by action type. - `getHistory(limit)` — Approval/rejection history. - `getStatus()` — Pending counts by action, stats. - `expireStaleRecommendations()` — Auto-expires pending recs older than 48 hours.

# 8. Implementation Roadmap

## Phase 1: Foundation — DONE

| DELIVERABLE | STATUS |
|---|---|
| D1 schema + migrations | Done |
| SoldComps API integration (every 15 min) | Done |
| PriceCharting API integration (daily) | Done |
| PSA population scraping (daily) | Done |
| Feature engineering pipeline | Done |
| LightGBM quantile regression training pipeline | Done |
| ONNX export + R2 upload | Done |
| Batch scoring CLI | Done |

## Phase 2: Intelligence — DONE

| DELIVERABLE | STATUS |
|---|---|
| Reddit sentiment pipeline (NER + classification via Workers AI) | Done |
| Sentiment rollup (24h/7d/30d, hourly cron) | Done |
| Anomaly detection (daily, before features) | Done |
| Conformal prediction intervals | Done |
| Volume-aware routing (high/medium/low buckets) | Done |
| Buy/sell decision API ( `POST /v1/evaluate` ) | Done |
| Hono/Workers API serving layer with auth | Done |

## Phase 3: Production — DONE

| DELIVERABLE | STATUS |
|---|---|
| React dashboard (market overview, card detail, search, evaluate, alerts) | Done |
| Pipeline health monitoring ( `/v1/system/health` , `ingestion_log` ) | Done |
| Model rollback system ( `POST /v1/system/rollback` ) | Done |
| Automated weekly retraining (GitHub Actions, Sunday 2 AM) | Done |
| Containerized ML training via Cloudflare Containers | Done — LightGBM trained on 16,144 observations, 42% MDAPE, 91.5% coverage |
| CI pipeline (typecheck, lint, Python syntax check) | Done |
| Price alerts system (spike, crash, viral, anomaly, new high/low) | Done |
| KV price cache with automatic invalidation | Done |

## Phase 4: Agentic Layer — DONE

| DELIVERABLE | STATUS |
|---|---|
| PriceMonitorAgent (anomaly + viral detection, every 15 min) | Done |
| MarketIntelligenceAgent (daily AI market briefing) | Done |
| CompetitorTrackerAgent (price gap scanning, every 6h) | Done |
| PricingRecommendationAgent (BUY/SELL/REPRICE with approval workflow) | Done |
| Agent REST proxy + WebSocket access | Done |
| Agent dashboard page in frontend | Done |

## Level 1: Dynamic Pricing — DONE

| DELIVERABLE | STATUS |
|---|---|
| Demand-aware pricing optimizer (inventory, velocity, competition, events, channel) | Done |
| Advanced evaluate endpoint ( `POST /v1/evaluate/advanced` ) | Done |
| Trade-in decision engine (BUY/DECLINE/HOLD with profit/risk/reasoning) | Done |
| Channel-specific pricing (in-store +5%, eBay +15%, online baseline) | Done |

## Remaining Work

| ITEM | PRIORITY | NOTES |
|---|---|---|
| Twitter/X sentiment integration | P2 | ~$500-1K/mo, filtered stream |
| GameStop internal data integration (trade-ins, inventory, foot traffic) | P1 | Requires internal DB access |
| A/B testing framework (model comparison) | P2 | Compare model versions on live traffic |
| Model drift detection + automated alerts | P2 | Track prediction accuracy over time; use $10+ price floor filter for meaningful MDAPE |
| CORS lockdown to GameStop domains | P0 | Currently `cors("*")` |
| GameStop SSO for dashboard | P1 | Production auth |
| eBay Marketplace Insights API access | P1 | Negotiate via retail partnership |
| TCGPlayer API integration | P2 | Requires application approval |
| Hierarchical Bayesian / GP models for low-volume | P3 | LightGBM with fallback is v1 |
| Temporal Fusion Transformer for high-volume | P3 | Research item |
| D1 archival strategy implementation | P2 | Needed at ~6 months |
| Level 2: Segment-based pricing (collector/flipper/casual/whale) | P2 | Requires PowerUp Rewards data integration |
| Level 3: Behavioral signals (search freq, watchlists, cart abandonment) | P3 | Legal review needed for public company |
| Daily feature snapshots for point-in-time ML training | P1 | Fixes lookahead bias in training data |
| Proper backtest simulation with separate offer stream | P2 | Current backtest uses target as decision input |

## 9. Competitive Landscape

| | PRICECHARTING | CARD LADDER (PSA) | ALT | |
|---|---|---|---|---|
| **Data sources** | eBay + own marketplace | 14 platforms | Multiple | SoldComps + PriceCharting + PSA pop + Reddit |
| **ML pricing** | Algorithmic smoothing (no ML) | Unknown | Likely gradient boosting | LightGBM quantile regression (707 cards, 42% MDAPE) + statistical fallback for new cards |
| **Sentiment** | None | None | Unknown | Reddit NLP (Workers AI) |
| **Update frequency** | Daily | Near real-time | Near real-time | Hybrid: 15-min ingestion + daily model + 15-min agent monitoring |
| **Uncertainty** | None | None | None | Conformal prediction intervals (p10-p90) |
| **Physical retail data** | No | No | No | **DEFERRED** — requires internal integration |
| **Anomaly detection** | Manual review | Unknown | Unknown | Automated (IQR outliers, data quality, viral detection) |
| **Autonomous agents** | No | No | No | 4 agents (monitor, intelligence, competitors, recommendations) |
| **Buy/sell decisions** | No | No | Unknown | NRV-based with margin targets + human approval workflow |

**Honest assessment:** v1 does not yet have GameStop's "unfair advantage" (internal store data). The moat exists but is not yet plumbed. What ships is a best-in-class ML pricing engine with uncertainty quantification and autonomous monitoring — competitive with or ahead of Alt/Card Ladder on technical sophistication, behind on data breadth until internal data is connected.

# 10. Cost Estimate (Cloudflare Stack)

## Monthly Operating Costs

| ITEM | COST/MONTH | NOTES |
|---|---|---|
| Cloudflare Workers Paid plan | $5 | Includes 10M requests/mo |
| Cloudflare D1 | ~$5-15 | Based on rows read/written |
| Cloudflare KV | ~$5-10 | Price cache + rate limiting |
| Cloudflare R2 | ~$5-10 | Model artifacts, data archive (no egress fees) |
| Cloudflare Queues | ~$1-5 | Two queues, moderate volume |
| Cloudflare Workers AI | ~$10-50 | Gemma 4 26B for reports + DistilBERT for sentiment |
| Durable Objects (4 agents) | ~$5-15 | Billed per request + storage |
| SoldComps API (Scale) | $59 | Primary market data |
| CardHedger API | $49 | Multi-platform data |
| PriceCharting (Legendary) | ~$25 | Historical/aggregated prices |
| PokemonPriceTracker (Business) | $99 | Graded Pokemon data |
| Cloudflare Browser Rendering | ~$5-10 | Reddit scraping via headless Chrome |
| GitHub Actions (retraining) | ~$0-10 | Free tier usually sufficient |
| MLflow (local) | $0 | Self-hosted |
| **Total** | **~$280-$360/month** | |

**Deferred costs (not in v1):** - Twitter/X API: ~$500-1K/mo - Cloudflare Containers (ML training): ~$5-20/mo (containerized Python pipeline, on-demand) - GameStop internal data pipeline: $0 (internal infra)

**vs. Palantir/vendor alternative:** $100K-500K+/year for a fraction of the functionality.

## 11. Key Risks & Mitigations

| RISK | SEVERITY | MITIGATION |
|---|---|---|
| eBay API access denied | High | SoldComps + CardHedger as fallbacks; negotiate via GameStop's retail partnership |
| Model accuracy insufficient for low-volume cards | Medium | Volume-aware routing, wide uncertainty bands, human review for <10 sales/quarter |
| Social sentiment is noisy/manipulable | Low | Weight sentiment low (1-3% of signal), use as regime detector not price predictor |
| Price history cold start | Medium | Backfill from PriceCharting CSV, SoldComps 365-day lookback |
| Fat-tailed prices break the model | Medium | Huber loss, log-transform, quantile regression, IQR outlier detection |
| eBay "Best Offer" bias (+15-25%) | Medium | Detect and discount by 20%, or exclude from training |
| Reddit scraping reliability | Low | Browser Rendering may break if old.reddit.com changes format; fallback to direct .json endpoint or Devvit app |
| **D1 10GB database limit** | Medium | Archive old price_observations to R2 at ~6 months; keep only recent windows in D1 |
| **Worker CPU time limits (30s)** | Medium | Batch D1 writes at 90 statements; chunk large operations; use Queues for async |
| **KV eventual consistency** | Low | 5-minute TTL on price cache; evaluate endpoint reads D1 directly |
| **Worker 128MB memory limit** | Low | Batch predictions loaded from R2 JSON; paginate for >100K cards |
| **Durable Object single-point-of-failure** | Low | Agents use hibernation + retry with backoff; state persists across restarts |
| **R2 model artifact corruption** | Medium | Versioned predictions in `models/versions/` ; full rollback system implemented |
| **Cron trigger overlap** | Low | Pipeline is ordered with 1-hour gaps; each stage is idempotent |
| **CORS currently open ( `*` )** | High | Must lock to GameStop domains before production |
| **Sub-$10 card sales create misleading MDAPE metrics** | Medium | Price floor filter ($10) in monitoring; training data should also be filtered |

## Appendix A: References & Data Sources

**APIs & Data:** - eBay Marketplace Insights API - SoldComps — eBay Sold Listings API - CardHedger Enterprise API - PriceCharting API - PSA Public API - GemRate — Aggregated Pop Reports - PokemonPriceTracker API - Reddit API Pricing 2026

**Research & Methodology:** - PriceCharting Methodology - CAIA: Trading Cards and the Price of Perfection - eBay Bans AI Agents — Feb 2026

## Appendix B: ML Training CLI

The `packages/ml-training` Python package provides these CLI commands:

| COMMAND | PURPOSE |
|---|---|
| `gmestart-export-features` | Export features + training data from D1 via HTTP API |
| `gmestart-train` | Train LightGBM quantile models |
| `gmestart-export` | Export trained models to ONNX format |
| `gmestart-backtest` | Walk-forward backtesting |
| `gmestart-score` | Batch score all cards, upload predictions to R2 |

**Automated retraining:** GitHub Actions workflow runs weekly (Sunday 2 AM UTC) via Cloudflare Containers: export features from D1 → train LightGBM in containerized Python pipeline → ONNX export → batch score → upload predictions to R2 → Worker picks up within 10 minutes. No external compute required.

## Appendix C: Frontend Pages

| ROUTE | COMPONENT | PURPOSE |
|---|---|---|
| `/` | MarketOverview | Action queue with market stats + recent alerts |
| `/search` | SearchBar | Card search with category filtering |
| `/card/:cardId` | CardDetail | Price history, predictions, sentiment for a card |
| `/evaluate` | EvaluateCard | Buy/sell/hold recommendation at a given price |
| `/alerts` | AlertsList | Alert triage with resolve controls |
| `/agents` | AgentDashboard | Real-time agent status via WebSocket |

Categories: All Cards, Pokemon, Baseball, Basketball, Football, MTG, Yu-Gi-Oh.

---

*Built with production experience from Exmplr (multi-agent AI platform over 562K+ clinical trials, RAG over 19.8M vectors) and ServeSys.AI (healthcare edge AI, 27K LOC Rust, 384 tests). The same architectural patterns — deterministic recipe routing, evaluation frameworks, quality gates, incremental pipelines — apply directly to collectibles pricing.*